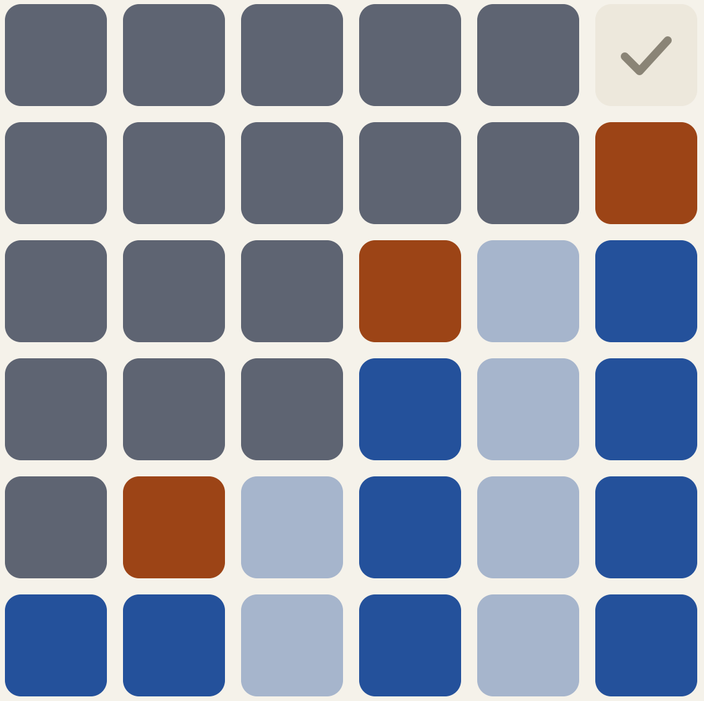
POST-TRAINING TUNING AND CONTINUOUS LEARNING

The system is designed to keep learning: we re-tune on demand.



Meta’s Muse Glimmer 30B is tuned on checked examples, and we promote a tuned model only after evaluation, under a rule fixed before scoring.

Why training alone doesn’t fix it

01

A model freezes when it ships

It never sees the tools, schemas and failures that appear after its release, so the same mistakes repeat.

02

Even a 30B model misses

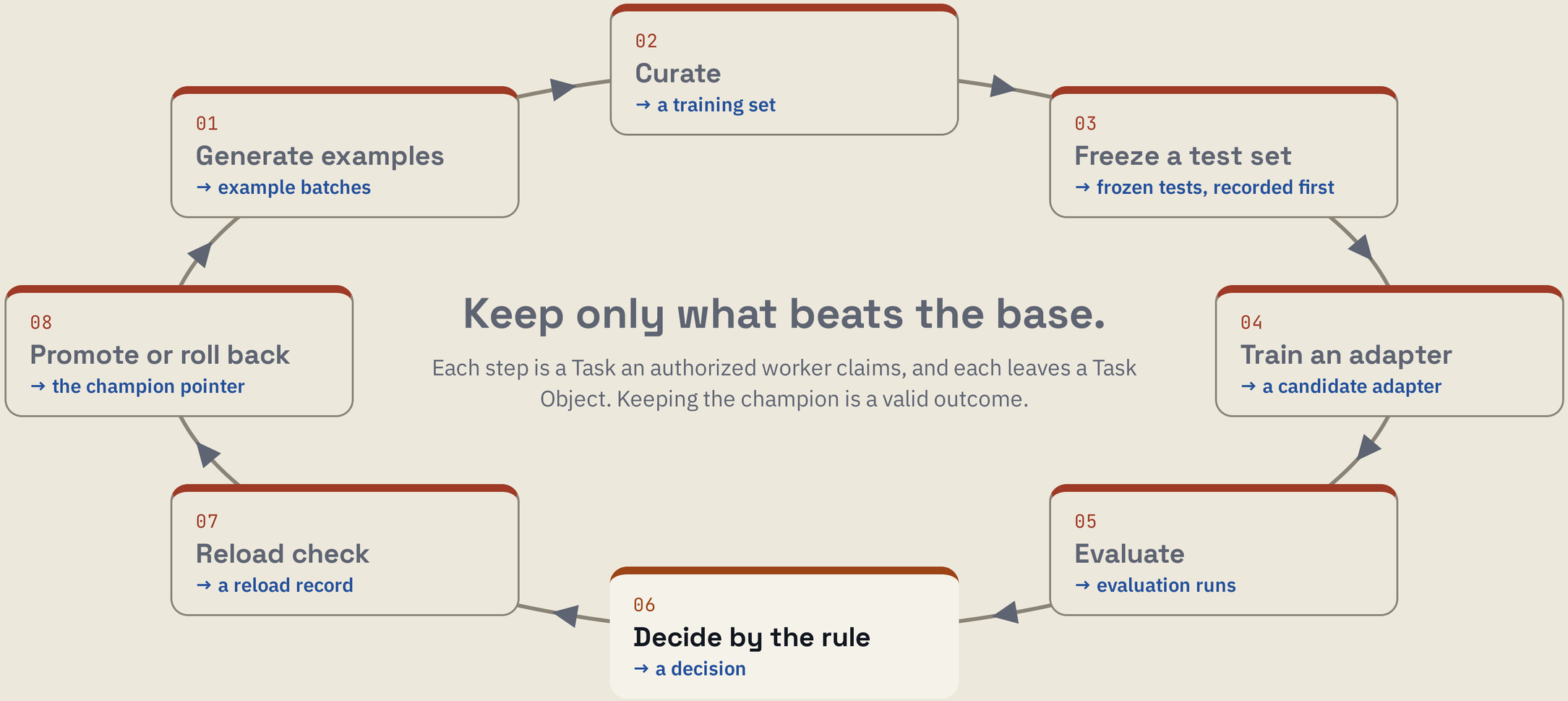
Unchanged, Muse Glimmer 30B answered 93.3% of a fresh tool-call test exactly right, and 96.7% of its replies were valid JSON.

THE FIX

Re-tune on demand, promote after evaluation

Tune on checked examples of the job, and keep a tuned model only when it beats its base on a test it never saw, under a rule fixed in advance.

The loop


01
Generate examples
→ example batches
02
Curate
→ a training set
03
Freeze a test set
→ frozen tests, recorded first
04
Train an adapter
→ a candidate adapter
05
Evaluate
→ evaluation runs
06
Decide by the rule
→ a decision
07
Reload check
→ a reload record
08
Promote or roll back
→ the champion pointer
Keep only what beats the base.
Each step is a Task an authorized worker claims, and each leaves a Task Object. Keeping the champion is a valid outcome.

Pre-registration and frozen tests

FIXED BEFORE ANY SCORE

FROZEN TEST
150 rows over tool families that training never sees, recorded once and never changed.

FRESH TEST
New rows each round, recorded before any model sees them.

THE PROMOTION RULE
Committed before the first training round, and pinned by a test.

NO LOOK-ALIKES
Fingerprints reject training rows that match a test row, its request or its template.

THE RULE DECIDES

Muse Glimmer 30B, 63 examples — Kept out

+5.3 points on the frozen test, but the 95% CI started at 0.0.

Muse Glimmer 30B, 511 examples — Promote

+6.0 points on a fresh test, with a 95% CI of +2.0 to +10.7, and 12 of 12 checks passed.

A guard written before any result stopped a real-looking gain, and passed a real one.

The 30B result

MUSE GLIMMER 30B · QLORA ON FREE NOTEBOOK GPUS

93.3% → 99.3%

Exact match on our 150-task tool-call evaluation, a fresh test recorded before scoring. Trained on 511 examples.

+6.0 points
95% CI +2.0 to +10.7

10 / 1
rows better / worse

12 / 12
promotion-rule checks

8 / 8
rows matched on reload

The rule said promote. Nothing has been promoted or admitted, and the adapter weights aren’t published.

Valid JSON rose from 96.7% to 100.0% and correct refusals from 97.3% to 99.3%, with 0 test rows matching the 511 training rows.

Learners: memory, paths and routing

Not all learning changes a model. Learner runs are ordinary Tasks: they read execution reports and write their advice as versioned Task Objects.

MEMORY

Similar past work

Each run is fingerprinted. Before new work, recall finds similar past runs and how they went, failures included.

`memory · minhash-lsh`

PATHS

What succeeds

Runs are grouped by category, template, phrasing and shim into scorecards. They show associations, not causes.

`path_statistics · wilson-grouping`

ROUTING

Which way to try

Weights suggest a shim, method, harness or model, keep exploring the others, and move only with enough evidence.

`routing_weights · beta-bandit`

MODEL WEIGHTS

The third layer: tuning on real runs

It takes only opted-in, checked, rights-cleared runs.

Advice only: never a permission, never a hidden agent. The learners are implemented and tested; their measured results so far come from our evaluation tasks.

Run it on free GPUs

FREE NOTEBOOK GPUS

A 30B model on two free T4s

Muse Glimmer 30B was tuned with QLoRA on Kaggle’s free notebook GPUs: 64 training steps in 83 minutes.

NO GPU

Check the plumbing first

A CPU backend runs every Task, report and decision, and records each step, without a GPU. Its numbers describe the plumbing only.

```
# one tick plans and runs one round; every Task is claimed, kept alive and recorded
$ PYTHONPATH=src:. python -m examples.continuous_tuning.cli tick \
   --root <runs> --base <base model> --base-path <weights> --backend hf
```

The guide, its tests and the CPU backend: `teleion-fabric-infrastructure-work-and-storage-engine/docs/continuous-tuning.md`

Safety and rollback

A round that fails any check leaves the champion where it was.

FROZEN TESTS

Held-out tool families

Training never sees the test's tools, and every test set is recorded once and never changed.

PRE-REGISTRATION

Recorded before scores

The rule and each fresh test are recorded before any model is scored on them.

CONTAMINATION CHECKS

No look-alikes

Fingerprints reject a training row that matches a test row, its request or its template.

RELOAD EQUIVALENCE

It must reload the same

The saved adapter, loaded onto a fresh copy of the base, must reproduce its replies exactly.

A RULE FIXED IN ADVANCE

Beat the base, break nothing

A gain whose lower bound is above zero, and no drop in valid output, refusals or general skills.

ROLLBACK

One pointer, every version kept

Adapters are never overwritten. Promotion moves one pointer, so rolling back moves it back.

Promotion changes no deployment, publication, admission or serving. Privacy is checked too: another key tried to read everything the earlier tuning rounds wrote, and all 303 reads were denied.

Limits: what these results cover

- Every result comes from our evaluations, one task per result: typed tool calls for Teleion Fabric’s own tools.
- Each configuration ran once, with one seed, so nothing measures the spread between runs.
- The 150-row tests are small: they can miss a real gain of a few points.
- Rounds start on demand, on recorded runs and our evaluations.
- Adapter weights stay private, and nothing has been promoted or admitted.
- The learners pass their tests on our evaluations.

Every kept-out round and every failure stays on the record.